PROJECT PROPOSAL

DESIGN AND IMPLEMENTATION OF HANGMAN GAME USING ASSEMBLY LANGUAGE

SUBMITTED BY:

Team Members:

1. Abdullah Ejaz Janjua (2023038)
2. Muhammad Ahmad Amjad (2023361)
3. Muhammad Haris (2023428)

SUMITTED TO:

Instructor:

Sir Zaheer Ahmed
FCSE
CE – 222 Lab

OUTLINE:

NOTE: This proposal outlines the initial plan for the project. However, minor modifications or adjustments may be made during the development phase based on implementation challenges and improvements identified during the process.

PROJECT DESCRIPTION:

The Hangman game is a classic word-guessing game where the player attempts to reveal a hidden word by guessing letters one at a time. For this project, we propose implementing Hangman entirely in **x86 Assembly Language**, simulating a complete text-based gaming experience within a DOS environment.

The game will prompt the user to guess letters of a randomly chosen word (from a pre-defined list). Each correct guess will reveal the letter's position(s) in the word, while incorrect guesses will decrease the number of remaining attempts. The game continues until the player either correctly guesses the full word or exhausts all available tries. The program will use keyboard input and display text output using interrupt-based I/O, string processing, and control flow features native to Assembly.

This project is intended to demonstrate deep understanding of Assembly-level programming by combining logic design, memory management, and user interaction in a low-level programming environment. Through this task, we aim to showcase how high-level functionalities like games can be built from scratch using fundamental computer architecture concepts.

GOALS AND OBJECTIVES:

1. To gain hands-on experience in **low-level Assembly programming** by creating a real-time application.
2. To strengthen our knowledge of **x86 architecture**, memory addressing, CPU registers, and system-level I/O operations.
3. To develop a deeper understanding of **control structures** (loops, conditionals, branching) and **string manipulation** in Assembly.

4. To explore **interrupt-driven input/output handling** (using INT 21h) for keyboard and screen interaction.
5. To manage **stack and memory segments** effectively for storing the hidden word, guessed letters, and game state.
6. To improve problem-solving, debugging, and logical reasoning by overcoming the challenges of writing an interactive program in Assembly.

TECHNICAL REQUIREMENTS AND TOOLS:

- **Programming Language:** x86 Assembly Language (8086 Instruction Set)
- **Assembler:**
  - Microsoft Macro Assembler (MASM)
- **Operating System:**
  - Windows/Linux/Mac (DOSBox-compatible)
- **Text Editor/IDE:**
  - Vs-Code

KEY FEATURES AND FUNCTIONALITIES:

- Display a welcome screen with basic instructions.
- Randomly select a word from a predefined word list stored in memory.
- Accept user input letter-by-letter via keyboard.
- Check if the guessed letter exists in the word:
  - If correct, reveal the letter in its correct positions.
  - If incorrect, reduce remaining attempts and update hangman status.
- Display used letters, remaining guesses, and game progress dynamically.

- Handle both win and loss scenarios, showing appropriate messages.
- Provide an option to replay or exit the game.

EXPECTED OUTCOMES:

- A working version of the Hangman game that runs inside a DOS environment, where the user can guess letters and try to figure out the hidden word.
- A complete Assembly language program that is properly written, organized, and well-commented so that others can understand how it works.
- Better understanding of how Assembly language works, especially in areas like:
- Taking input from the keyboard and showing text on the screen
- Using memory and storing game data like the hidden word and guessed letters
- Using loops and conditions to control the flow of the game
- Experience using real tools like MASM to write and run Assembly programs.
- A clear and easy-to-follow project report that explains:
    1. How we designed the game
    2. What problems we faced and how we solved them
    3. What we learned while making the game
- More confidence in writing low-level code and solving real problems using Assembly language.